Sistema Kiosk Protegido com Operação Autônoma 24/7: Arquitetura, Proteção Multi-Camada e Aceleração por Inteligência Artificial

Versão 1.0.0 — Produção | Fase v1.0 concluída em 16 de abril de 2026

Documento público — dados sensíveis de configuração, protocolo e infraestrutura omitidos intencionalmente.

AUTORES E CRÉDITOS

Cleyton Pedroza

Criador dos componentes fundamentais do sistema. Responsável pela concepção da arquitetura base, desenvolvimento do protocolo de comunicação proprietário, criação da infraestrutura de servidores, criptografia e design original dos fluxos de operação.

Leandro Batista

Responsável pelo aperfeiçoamento, implementação prática e desenvolvimento de hardware. Transformou a base arquitetural em produto funcional em produção — implementando os drivers, o sistema de bloqueio físico multi-camada, a automação de build e os processos de recuperação autônoma. Fase v1.0 concluída em 7 dias com auxílio de IA.

RESUMO

Este paper descreve a arquitetura, implementação e validação de um sistema completo de Kiosk/Loja virtual desenvolvido para operação autônoma ininterrupta (24/7) sem necessidade de intervenção manual. O sistema integra cinco componentes de software — quatro em produção e um em desenvolvimento — comunicando-se via protocolo serial proprietário. São implementadas quatro camadas de proteção: física, de software, de sistema e operacional. A fase v1.0 foi entregue em 7 dias por um único desenvolvedor, com aceleração significativa proporcionada pelo uso de ferramentas de Inteligência Artificial — uma tarefa que, no modelo tradicional com equipe, demandaria entre 30 e 60 dias.

Palavras-chave: *Kiosk, Protocolo Serial Proprietário, Bloqueio Físico, Automação, Linux, Python, Inteligência Artificial, Desenvolvimento Acelerado.*

1. INTRODUÇÃO

Sistemas de Kiosk são amplamente utilizados em aplicações comerciais que exigem operação contínua e segura sem supervisão humana direta. O desafio fundamental nesse domínio é garantir que o sistema opere de forma autônoma, resista a tentativas de manipulação física e digital, e se recupere automaticamente de falhas — tudo isso com latência mínima e sem interrupção do serviço.

O sistema foi concebido com uma base técnica sólida pré-existente — arquitetura, protocolos e criptografia desenvolvidos por Cleyton Pedroza — e implementado em sua totalidade funcional por Leandro Batista em 7 dias, com auxílio de ferramentas de IA. Este resultado representa uma redução de 75–90% no tempo de desenvolvimento estimado pelo modelo tradicional (30–60 dias com equipe).

> ***Nota:*** *Detalhes de configuração, caminhos de instalação, mensagens do protocolo e identificadores internos foram omitidos intencionalmente. O sistema encontra-se em fase de stage/teste em campo.*

2. ARQUITETURA DO SISTEMA

2.1 Componentes Principais

#	Componente	Função	Status
1	Driver Principal	Core do protocolo serial proprietário (botões, sensores, periféricos)	✔ **Produção**
2	Updater	Atualização automática com rollback	✔ **Produção**
3	Launcher	Interface Kiosk fullscreen isolada	✔ **Produção**
4	System Lock	Bloqueio físico de dispositivos de entrada	✔ **Produção**
5	Game Module	Jogos Interativos	⚙ **Em desenvolvimento**

2.2 Ambiente de Execução

Categoria	Tecnologia
Sistema Operacional	Linux
Gerenciador de Interface Gráfica	Minimalista (sem ambiente de desktop)
Gerenciador de Serviços	Daemon de Serviços do Sistema
Linguagem	Python 3
Compilação / Distribuição	Compilador de Binários
Versionamento	Git
Automação de Build	Ferramenta de Build Automatizado
Integridade	Checksum criptográfico (algoritmo não divulgado)

2.3 Sequência de Boot Determinística

```
Estágio 1 → Boot do Sistema Linux
Estágio 2 → Gerenciador de Interface Gráfica carrega (ambiente minimalista)
Estágio 3 → Scripts de autostart executam
Estágio 4 → Driver Principal inicia (protocolo de comunicação)
Estágio 5 → Updater inicia (após intervalo controlado)
Estágio 6 → Launcher Kiosk inicia (após intervalo controlado)
Estágio 7 → System Lock inicia via serviço do daemon
Estágio 8 → Monitoramento contínuo ativo em todos os componentes
```

3. PROTOCOLO DE COMUNICAÇÃO PROPRIETÁRIO

Protocolo serial proprietário desenvolvido por Cleyton Pedroza para padronizar a troca de mensagens entre os componentes e os periféricos de hardware. Opera sobre interface serial física e define comandos para controle de sensores físicos, periféricos de entrada de valores e iluminação.

> ***Nota de segurança:*** *A estrutura de mensagens, identificadores de comandos, endereçamento e parâmetros do protocolo são informações proprietárias e não são divulgados neste documento.*

Garantias implementadas: integridade por checksum por mensagem, timeout com prevenção de deadlocks, reconexão automática serial, monitoramento contínuo de componentes e isolamento total do sistema de bloqueio físico.

4. DRIVER PRINCIPAL

Núcleo do sistema, responsável pela implementação completa do protocolo de comunicação proprietário e gerenciamento bidirecional de todos os periféricos de hardware.

Responsabilidade	Detalhe
Protocolo proprietário	Implementação completa com validação de checksum por mensagem
Sensores físicos	Gerenciamento de botões de interação
Periférico de valores	Controle e leitura de estado
Iluminação	Controle do sistema de iluminação
Simulação de eventos	Geração de eventos de entrada para o sistema de jogos
Health check	Verificação contínua de todos os componentes integrados
Auditoria	Logging completo de operações

Compilado em binário nativo para Linux — código-fonte não exposto no ambiente de produção. Deploy remoto automatizado com rollback automático para versão anterior em caso de falha.

5. SISTEMA DE BLOQUEIO FÍSICO (SYSTEM LOCK)

5.1 Motivação

Em sistemas Kiosk de uso público, um vetor de ataque comum é a inserção de dispositivos de entrada externos para obter acesso ao sistema operacional subjacente. O System Lock mitiga esse vetor com seis camadas independentes de bloqueio no kernel Linux — garantindo que, mesmo que uma camada seja contornada, as demais continuem ativas.

5.2 Arquitetura em 6 Camadas Independentes

Camada 1 — Bloqueio Direto via Sistema de Arquivos (Prioridade Máxima)

Remoção completa de permissões de acesso ao dispositivo no sistema de arquivos Linux, combinada com desabilitamento via interface de controle do kernel. Maior prioridade e menor latência.

Camada 2 — Bloqueio via Camada Gráfica

Desabilitamento do dispositivo na camada de servidor gráfico do sistema. Garante que dispositivos com permissões de sistema não possam enviar eventos para aplicações gráficas.

Camada 3 — Remoção de Módulos do Kernel

Descarregamento seletivo dos módulos do kernel responsáveis pelo suporte a dispositivos de interface humana. Impede que o kernel processe eventos do dispositivo na raiz do suporte ao hardware.

Camada 4 — Bloqueio de Interface de Dispositivo (Wireless / Bluetooth)

Bloqueio específico para dispositivos que comunicam via interface de entrada de baixo nível, cobrindo wireless RF, dongles 2.4GHz e Bluetooth. Remoção de permissões nos nós de dispositivo correspondentes.

Camada 5 — Captura Exclusiva via Chamada de Sistema

Chamada de sistema para captura exclusiva do dispositivo. Apenas o processo detentor recebe os eventos, impedindo que qualquer outra aplicação processe entradas do dispositivo bloqueado.

Camada 6 — Regras Persistentes de Gerenciamento de Dispositivos

Regras no sistema de gerenciamento de dispositivos que garantem bloqueio persistente após reinicializações e aplicação automática a qualquer novo dispositivo inserido.

5.3 Dispositivos Cobertos

Tipo	Tecnologia	Status
Teclados USB	Todos os tipos e fabricantes	**Bloqueado**
Teclados PS2	Portas legadas	**Bloqueado**
Teclados Wireless RF	2.4GHz, proprietários	**Bloqueado**
Teclados Bluetooth	Todos os profiles	**Bloqueado**
Receptores Universais	Multi-dispositivo	**Bloqueado e validado**
Dispositivos de interface humana	Genéricos	**Bloqueado**

5.4 Monitoramento e Hotplug

Loop de monitoramento com verificação a cada 1 segundo via análise de timestamps do sistema de arquivos. Ao detectar novo dispositivo de entrada, todas as 6 camadas são aplicadas imediatamente.

Métrica	Valor
Latência máxima de detecção	1 segundo
Latência de bloqueio após detecção	Imediata (síncrona)
Cobertura	Qualquer dispositivo de entrada conectado após o boot

6. INTERFACE KIOSK (LAUNCHER)

Implementa o princípio de mínima exposição ao sistema operacional:

Proteção	Mecanismo
Fullscreen lock	Não pode ser minimizado ou redimensionado
Sem acesso ao SO	Sem terminal, sem gerenciador de arquivos
Process protection	Processo não pode ser encerrado pelo usuário
Auto-restart	Recuperação automática em menos de 3 segundos
Cursor invisível	Experiência Kiosk pura
Input validation	Apenas entradas autorizadas pelo protocolo são processadas

Subsistemas integrados: detecção automática de display com suporte multi-monitor, gerenciamento de rede com reconexão automática, monitoramento de processos com controle por lista de permissões e motor de análise preditiva para detecção antecipada de falhas.

7. SISTEMA DE ATUALIZAÇÃO (UPDATER)

1. **Verificação de versão** — Comparação entre versão local e versão disponível no servidor remoto.
2. **Download seguro** — Transferência via canal criptografado com verificação de autenticidade.
3. **Backup da versão atual** — Preservação completa antes de qualquer modificação.
4. **Validação de integridade** — Checksum criptográfico do artefato antes da instalação.
5. **Instalação silenciosa** — Sem interação do usuário e sem interrupção do serviço.
6. **Teste pós-instalação** — Verificação de funcionamento do componente atualizado.
7. **Limpeza** — Remoção de arquivos temporários.
8. **Rollback automático** — Em caso de falha em qualquer passo, restauração da versão anterior.

8. CAMADAS DE PROTEÇÃO — VISÃO CONSOLIDADA

```
┌──────────────────────────────────────────────────┐
│ CAMADA 4: OPERACIONAL │
│ Monitoramento contínuo · Recuperação automática │
│ Logs de auditoria · Health check · Alertas │
├──────────────────────────────────────────────────┤
│ CAMADA 3: SISTEMA │
│ Boot seguro · Atualizações automáticas │
│ Rollback · Validação de integridade · Daemon │
├──────────────────────────────────────────────────┤
│ CAMADA 2: SOFTWARE │
│ Tela preta Kiosk · Sem terminal acessível │
```

```
│ Cursor invisível · Restart automático │
├──────────────────────────────────────────────────────┤
│ CAMADA 1: FÍSICA │
│ Bloqueio USB · PS2 · Wireless · Bluetooth │
│ 6 métodos independentes · Detecção hotplug 1s │
└──────────────────────────────────────────────────────┘
```

9. O PAPEL DA INTELIGÊNCIA ARTIFICIAL NO DESENVOLVIMENTO

30–60
dias · modelo tradicional
3–5 pessoas

7
dias · com IA como parceira
1 pessoa

Área	Contribuição da IA
Implementação de drivers	Estrutura base gerada e validada iterativamente
Debug	Identificação e resolução de problemas em tempo real
Padronização	Consistência entre os 5 componentes mantida automaticamente
Documentação	Geração e organização da documentação técnica
Arquitetura	Validação de decisões e sugestão de padrões
Testes	Geração de casos de teste e análise de cobertura

O uso de IA não substitui o conhecimento técnico — ele amplifica a capacidade de execução. A base técnica sólida pré-existente foi o fator habilitador fundamental. *A profissão não foi substituída — ela foi comprimida e amplificada.*

10. VALIDAÇÃO E STATUS EM CAMPO

O sistema encontra-se em fase de **stage/teste em campo** com cliente. Cenários validados:

- ✔ Boot completo do sistema sem interação manual
- ✔ Bloqueio de teclado USB após hotplug
- ✔ Bloqueio de receptor wireless universal
- ✔ Protocolo de comunicação com System Lock ativo simultaneamente
- ✔ Atualização remota sem interrupção do serviço
- ✔ Recuperação automática após falha simulada
- ✔ Operação contínua em janela de 24 horas

11. STACK TECNOLÓGICO

Categoria	Tecnologia
Linguagem	Python 3
Sistema Operacional	Linux
Gerenciador de Interface Gráfica	Minimalista
Gerenciador de Serviços	Daemon de Serviços do Sistema
Comunicação Serial	Interface serial física (protocolo proprietário)
Simulação de Eventos	Simulador de Eventos de Entrada
Compilação	Compilador de Binários
Versionamento	Git
Automação de Build	Ferramenta de Build Automatizado
Criptografia / Integridade	Checksum criptográfico (algoritmo não divulgado)

12. CONCLUSÃO

Este paper apresentou a arquitetura e implementação de um sistema Kiosk com operação autônoma 24/7, proteção multi-camada e automação completa de ciclo de vida. O sistema demonstra que é possível construir infraestrutura de software embarcado robusta e de produção combinando três elementos fundamentais:

1. **Arquitetura bem definida como fundação** — concepção técnica sólida que define protocolos, fluxos e interfaces (contribuição de Cleyton Pedroza)
2. **Implementação focada com conhecimento de domínio** — transformação da arquitetura em produto funcional, testado e em produção (contribuição de Leandro Batista)
3. **IA como acelerador de execução** — compressão do ciclo de 30–60 dias para 7 dias, mantendo qualidade técnica

✔ **Fase v1.0 concluída em 16/04/2026.** Sistema em fase de stage/teste em campo com cliente.

REFERÊNCIAS

1. KIOSK INDUSTRY ASSOCIATION. *Best Practices for Unattended Kiosk Deployment and Security*. 2023. Disponível em: https://kioskindustry.org
2. CHEN, M.; TWOREK, J. et al. *Evaluating Large Language Models Trained on Code*. OpenAI, 2021. arXiv:2107.03374.
3. LINUX KERNEL DOCUMENTATION. *Input Subsystem — Event Interface*. The Linux Kernel Archives. Disponível em: https://www.kernel.org/doc/html/latest/input/input.html
4. FREEDESKTOP.ORG. *Dynamic Device Management — Linux Kernel Documentation*. 2024. Disponível em: https://www.kernel.org/doc/html/latest/driver-api/usb/index.html
5. THE LINUX DOCUMENTATION PROJECT. *Gerenciamento de Interface Gráfica em Ambientes Embarcados*. Disponível em: https://tldp.org
6. PYTHON SOFTWARE FOUNDATION. *Python Deployment and Distribution*. Disponível em: https://docs.python.org/3/distributing/
7. CORBET, J.; RUBINI, A.; KROAH-HARTMAN, G. *Linux Device Drivers*, 3rd ed. O'Reilly Media, 2005.

Sistema Kiosk Protegido e Automatizado · Versão 1.0.0 — Produção · Fase v1.0 concluída em 16/04/2026

Documento público — informações sensíveis de configuração e protocolo omitidas intencionalmente.